Assignment 1: Network and Web Server Performance

CS-452 Data Centers

tsazei01

October 2026

Setup

All experiments ran on four CloudLab nodes in the Utah cluster, running Ubuntu 22.04 (kernel 5.15) and connected over a private 10.10.1.0/24 network. The server nodes (node0, node2) have 48 CPUs each.

Node	Role
node0	iperf3 server, nginx server (single and multi node)
node1	iperf3 client, vegeta attacker (single and multi node)
node2	nginx server (multi node)
node3	vegeta attacker (multi node)

Network. iperf3 3.9 in UDP mode, node1 to node0, 10 s per run, offered bandwidth from 100 to 1000 Mbit/s in steps of 100, 10 runs per step.

Web server. nginx with `worker_processes 1`, serving a static page. Before every run the attack scripts check over SSH that exactly one worker process exists. Load was generated with vegeta 12.12.0, an open-loop generator: it sends at the requested rate regardless of how fast responses come back. Each run lasts 30 s, followed by a 5 s cooldown, with 10 runs per rate. While each run is active, `pidstat` samples the CPU usage of the nginx worker once per second. Because a single worker runs on a single core, 100% means that core is fully busy. The CPU figures below average only the samples that fall inside the attack window, matched by timestamp; the idle seconds before and after are excluded.

Multi node. Both attackers start at the same scheduled time. Each sends half of the aggregate rate, alternating requests between node0 and node2, so each server receives half of the aggregate rate. node1's results were saved as vegeta JSON reports. node3's JSON reports were not copied off the node, so its runs are taken from the text reports in its experiment logs, which contain the same counters and latency percentiles. Server CPU in the multi-node experiment comes from node1's monitors only, which sampled the same two servers.

Statistics. Every value is the mean over 10 runs, given as mean ± 95% confidence interval (Student t, 9 degrees of freedom). The *saturation point* is the last rate before throughput first falls below 95% of the offered load. *Capacity under a p99 target* is the last rate before the mean p99 latency first exceeds that target. The analysis code is in `analysis/`.

Network

Offered (Mbit/s)	Goodput (Mbit/s)	Loss (%)	Jitter (ms)	Client CPU (%)	Server CPU (%)
100	100.0 ± 0.0	0.000 ± 0.000	0.0059	6.4	8.0
200	200.0 ± 0.0	0.000 ± 0.000	0.0132	11.0	17.0
300	300.0 ± 0.0	0.000 ± 0.000	0.0099	16.1	24.7
400	400.0 ± 0.0	0.000 ± 0.000	0.0079	21.3	32.4
500	500.0 ± 0.0	0.000 ± 0.000	0.0062	26.4	40.9
600	600.0 ± 0.0	0.000 ± 0.000	0.0052	31.7	48.7
700	700.0 ± 0.0	0.000 ± 0.000	0.0048	36.5	57.9
800	799.9 ± 0.0	0.002 ± 0.003	0.0040	41.6	62.1
900	899.9 ± 0.0	0.002 ± 0.002	0.0033	47.0	69.4
1000	934.0 ± 34.3	6.593 ± 3.427	0.0042	50.5	71.4

Goodput is the bandwidth the server actually received: sent × (1 − loss). Up to 700 Mbit/s no packets were lost. At 800 and 900 Mbit/s loss is about 0.002%. At 1000 Mbit/s the sender still sends 1000 Mbit/s, but the server receives on average only 934 ± 34 Mbit/s and loss rises to 6.6 ± 3.4%. The runs at 1000 Mbit/s split into two groups: three were essentially lossless (999.9 Mbit/s received, under 0.01% loss) and seven lost between 7% and 13%. A run that delivers 999.9 Mbit/s of UDP payload cannot be on a 1 GbE link, whose payload ceiling with 1448-byte datagrams is about 956 Mbit/s once Ethernet, IP and UDP overhead is counted. The link is therefore faster than 1 Gbit/s and was not saturated by this test. The loss at 1000 Mbit/s more likely comes from the receiving host, for example the socket buffer overflowing when the single-threaded iperf3 server falls behind; iperf3 does not report where packets were dropped. Jitter stays below 0.02 ms at every rate.

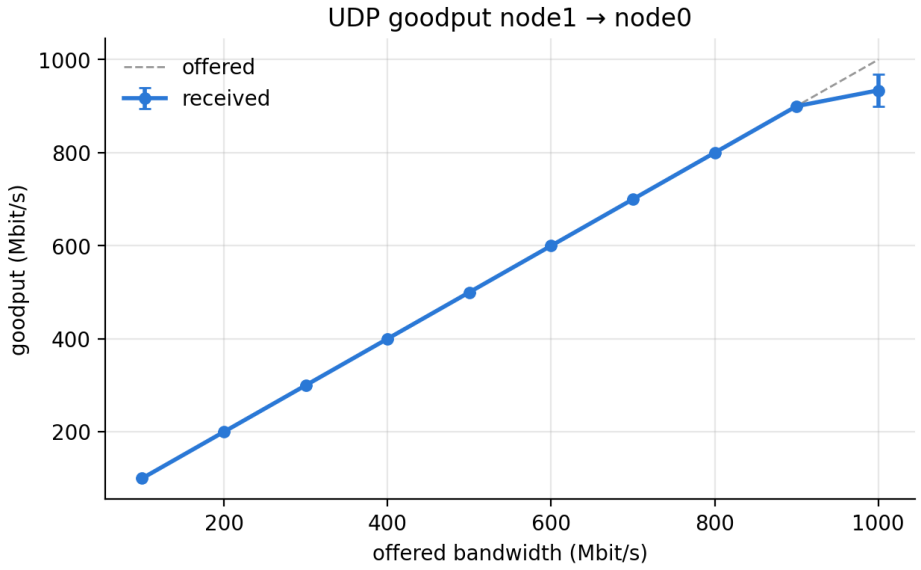


Figure 1: UDP goodput against offered bandwidth. The dashed line is goodput equal to offered.

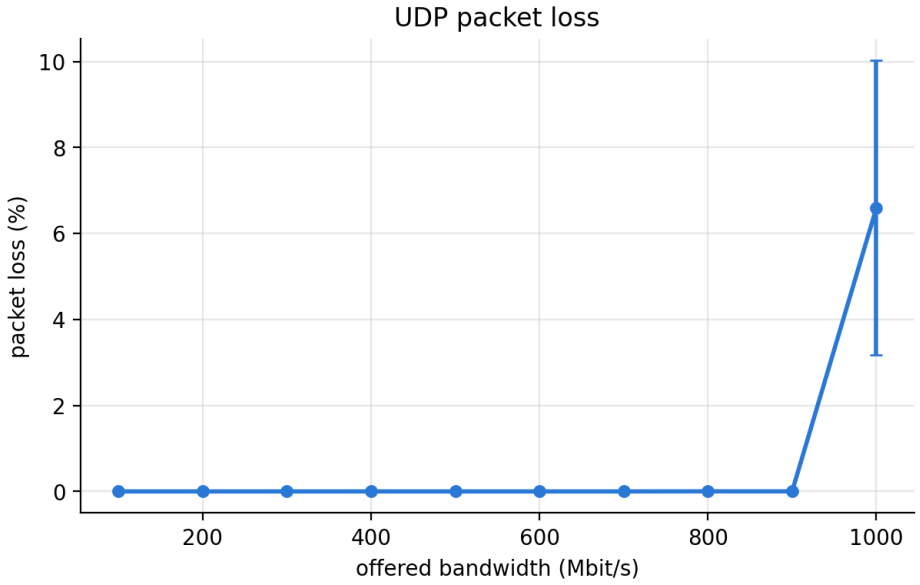


Figure 2: UDP packet loss against offered bandwidth.

Single node

Offered (req/s)	Sent (req/s)	Throughput (req/s)	Success (%)	p50 (ms)	p99 (ms)	node0 CPU (%)
1000	1000	1000 ± 0	100.0	0.27	0.59	11.3 ± 0.8
2000	2000	2000 ± 0	100.0	0.30	0.77	21.7 ± 0.7
3000	3000	3000 ± 0	100.0	0.35	0.91	31.8 ± 1.8
4000	4000	4000 ± 0	100.0	0.31	1.00	43.3 ± 0.5
5000	5000	5000 ± 0	100.0	0.24	0.89	55.6 ± 0.4
6000	6000	6000 ± 0	100.0	0.26	0.77	65.1 ± 2.1
7000	7000	7000 ± 0	100.0	0.21	0.75	48.7 ± 0.4
8000	8000	8000 ± 0	100.0	0.24	0.79	54.8 ± 0.4
9000	9000	9000 ± 0	100.0	0.39	1.76	60.7 ± 0.3
10000	10000	10000 ± 0	100.0	0.49	12.51	66.5 ± 0.5
12000	12000	12000 ± 0	100.0	0.44	24.42	59.3 ± 0.3
14000	13999	13998 ± 1	100.0	5.26	32.92	67.0 ± 0.3
16000	15999	15996 ± 1	100.0	8.70	58.54	76.2 ± 0.5
18000	17999	17995 ± 1	100.0	10.07	94.18	84.6 ± 0.5
19000	18942	15839 ± 2379	99.9	149.19	1121.29	91.8 ± 2.1
20000	15727	10803 ± 3494	70.2	471.79	3314.75	75.8 ± 5.5

Sent is the rate vegeta actually achieved. *Throughput* counts successful responses per second.

Throughput. node0 served every request on time up to 18000 req/s, so the saturation point is 18000 req/s. 19000 req/s is the boundary: five of the ten runs held about 18990 req/s and the other five collapsed to about 12700 req/s, which explains the wide interval. At 20000 req/s throughput fell to 10803 req/s, only 70% of requests succeeded, and vegeta itself could send just 79% of the requested rate, because its workers were stuck on stalled connections. The failed requests were connection resets, EOFs, connections refused, and local `address already in use` errors on the client, which mean the attacker ran out of ephemeral ports.

Latency. p99 stays at or below 1 ms up to 8000 req/s, reaches 12.5 ms at 10000 req/s and 94 ms at 18000 req/s, then jumps to 1.1 s at 19000 req/s. The median starts rising at 14000 req/s (5.3

ms). This means requests begin to queue well before throughput drops: from about 9000 req/s the worker no longer clears every burst before the next one arrives.

Capacity under a p99 target. 8000 req/s for p99 ≤ 1 ms, 9000 req/s for p99 ≤ 10 ms, and 18000 req/s for p99 ≤ 100 ms.

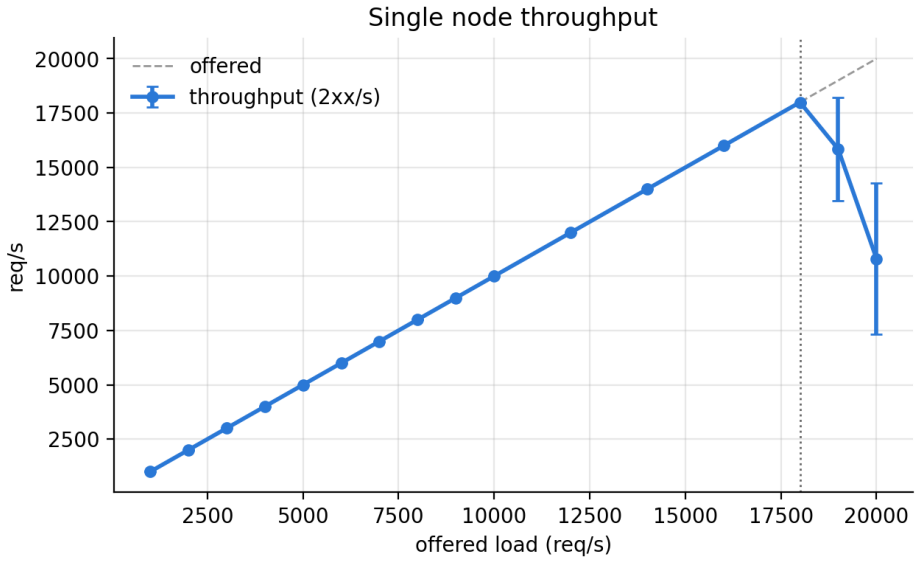


Figure 3: Single-node throughput. The dashed line is throughput equal to the offered load; the dotted line marks the saturation point.

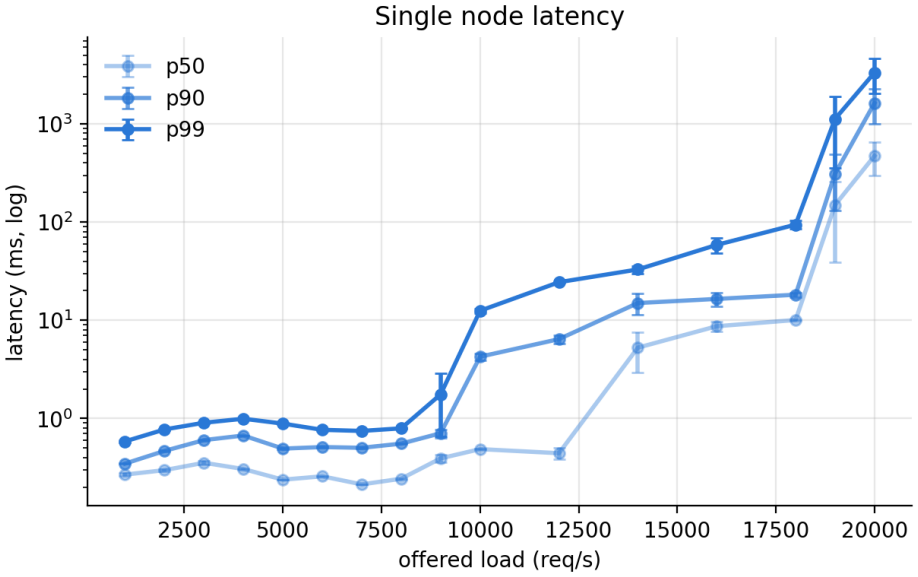


Figure 4: Single-node latency percentiles (log scale).

Bottleneck. During a 19000 req/s run the worker sits at 98 to 100% CPU for the whole attack, so the single nginx worker's core is the bottleneck. The network is not: response bodies at 19000 req/s amount to 93 Mbit/s, about 10% of the 900 Mbit/s that iperf3 carried without loss, and the link itself is faster than that. HTTP headers are not included in that figure, but even several hundred bytes of headers per response would leave the link far from full. About 60% of the

worker's CPU time is kernel time (`%system`), which is expected for a server whose work is mostly accepting connections and moving small buffers through sockets.

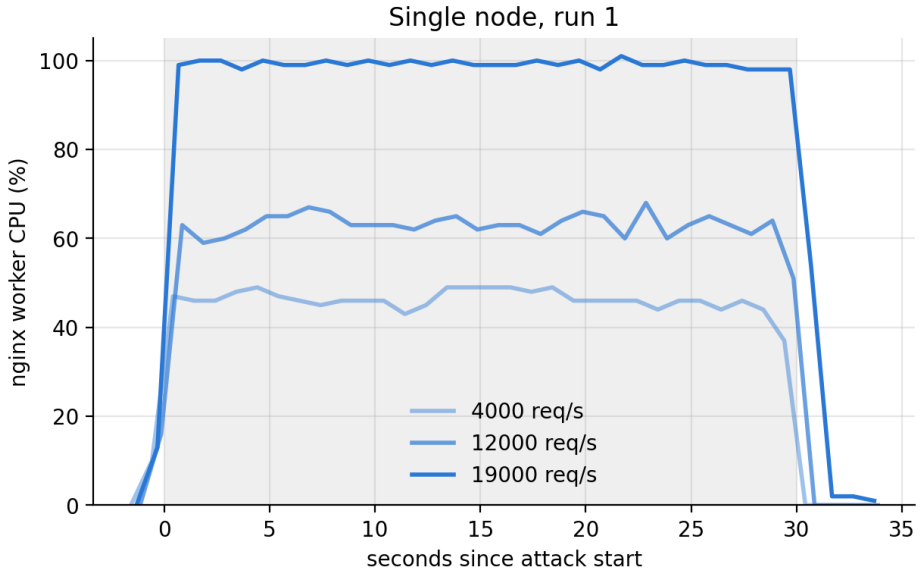


Figure 5: Per-second CPU of the nginx worker during run 1 at three rates. The shaded band is the 30 s attack.

CPU cost per request falls in steps. CPU does not grow linearly with load. It drops from 65.1% at 6000 req/s to 48.7% at 7000 req/s, and again from 66.5% at 10000 req/s to 59.3% at 12000 req/s. CPU per 1000 req/s goes from about 11% (up to 6000 req/s) to about 6.8% (7000 to 10000 req/s) to about 4.8% (12000 to 18000 req/s). The same drop appears in the multi-node experiment at the same load per server, so it is a repeatable property of the machines, not noise. `pidstat` reports CPU as a share of time, so if the core runs at a higher clock frequency, the same work takes a smaller share. A frequency step under rising load would produce exactly this pattern. More requests handled per `epoll` wakeup at higher rates would also lower the per-request cost. CPU frequency was not recorded, so this explanation is a hypothesis and was not verified.

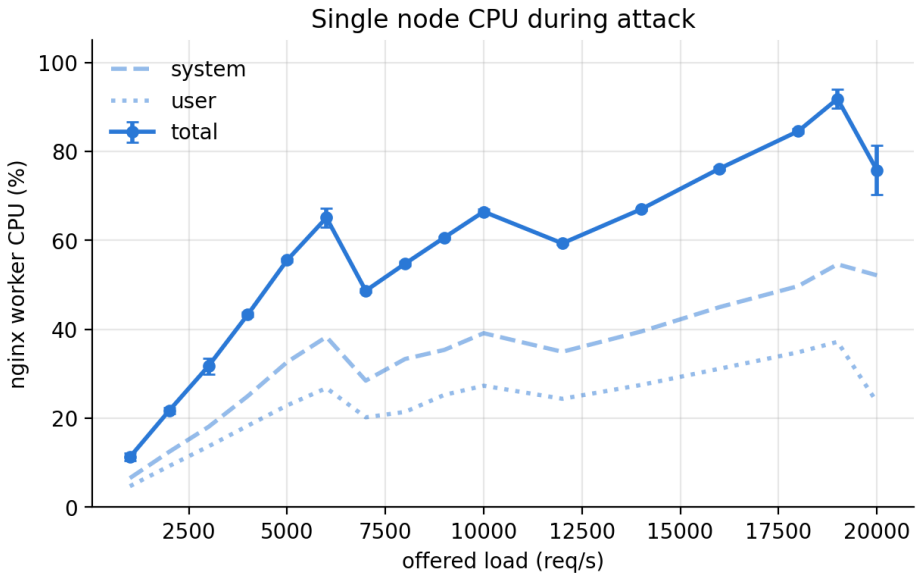


Figure 6: nginx worker CPU on node0 during the attack. Dashed and dotted lines split the total into kernel and user time.

Multi node

Offered (req/s)	Sent (req/s)	Throughput (req/s)	Success (%)	Mean (ms)	p99 (ms)	node0 CPU (%)
2000	2000	2000 ± 0	100.0	0.27	0.59	11.1 ± 0.8
4000	4000	4000 ± 0	100.0	0.29	0.75	22.9 ± 0.3
6000	6000	6000 ± 0	100.0	0.32	0.81	32.5 ± 0.7
8000	8000	8000 ± 0	100.0	0.34	0.88	43.5 ± 0.8
10000	10000	10000 ± 0	100.0	0.28	0.78	55.5 ± 0.6
12000	12000	12000 ± 0	100.0	0.29	0.75	65.1 ± 2.1
14000	14000	14000 ± 0	100.0	0.29	0.75	49.0 ± 0.3
16000	16000	16000 ± 0	100.0	0.33	0.82	55.7 ± 0.4
18000	18000	18000 ± 0	100.0	0.44	1.94	61.8 ± 0.4
20000	20000	20000 ± 0	100.0	1.44	14.48	68.4 ± 0.7
24000	24000	24000 ± 0	100.0	1.86	23.62	60.5 ± 0.4
28000	27999	27996 ± 2	100.0	6.46	34.88	67.6 ± 0.5
32000	31997	31990 ± 1	100.0	10.96	46.68	75.9 ± 0.7
36000	35995	35988 ± 2	100.0	14.43	62.83	83.6 ± 0.6
38000	37995	37987 ± 3	100.0	16.97	96.61	89.9 ± 1.0
40000	38566	26202 ± 601	96.6	414.53	4370.11	92.2 ± 2.4
42000	37352	28664 ± 2860	89.7	543.02	6292.22	87.7 ± 2.6
44000	36991	26145 ± 2954	84.1	602.60	6970.94	86.3 ± 2.3

Offered, sent and throughput are summed over both attackers. Mean latency is weighted by request count. vegeta reports only summary percentiles, and percentiles from two reports cannot be combined exactly, so the p99 column takes the worse of the two attackers. It is an upper bound on the true combined p99.

Throughput. The two servers together served the full offered load up to 38000 req/s (37987 ± 3 req/s), so the saturation point is 38000 req/s. At 40000 req/s throughput fell to 26202 req/s with 96.6% success, and it stayed between 26000 and 29000 req/s at 42000 and 44000 req/s. As in the

single-node case, the failures were connection resets, EOFs and client timeouts, plus ephemeral port exhaustion on the attackers.

Latency. Worst-attacker p99 is below 1 ms up to 16000 req/s, 14.5 ms at 20000 req/s, 96.6 ms at 38000 req/s, and several seconds past saturation. Capacity is 16000 req/s for p99 ≤ 1 ms, 18000 req/s for p99 ≤ 10 ms, and 38000 req/s for p99 ≤ 100 ms.

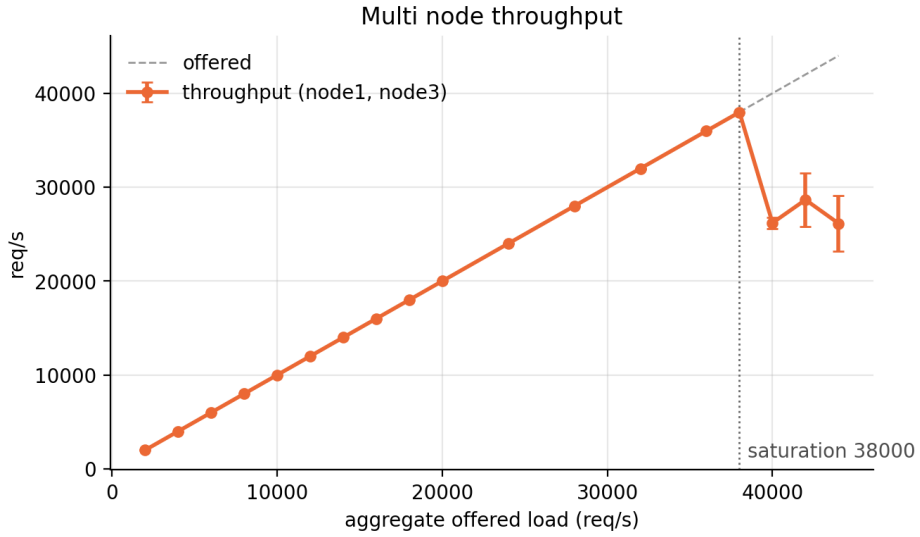


Figure 7: Multi-node aggregate throughput.

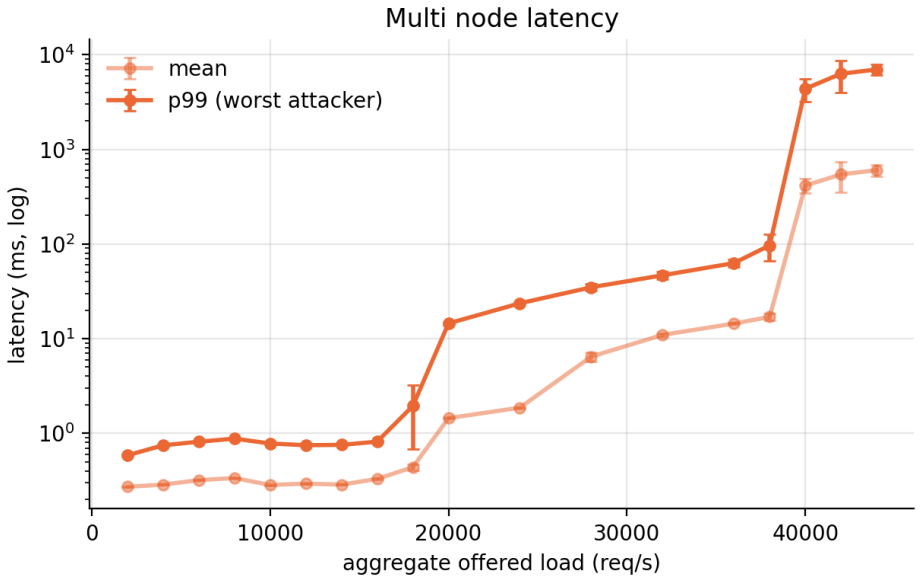


Figure 8: Multi-node latency (log scale). p99 is the worse of the two attackers.

The two servers were not configured identically. Every successful response from node0 in the single-node experiment was 612 bytes. The page in `src/app/index.html` is 22 bytes, and 612 bytes matches Ubuntu's default nginx welcome page. In the multi-node experiment both attackers alternate requests between the two servers, and both report exactly 317 bytes per successful response, which is (612 + 22) / 2. So node2 served the 22-byte application page, while

node0 served the default page in both experiments. The single-node results therefore describe nginx serving the 612-byte default page, and the multi-node results describe one server of each kind.

Load balance. Round-robin splits requests evenly, but the servers did not do equal work. Up to 12000 req/s, node0 used 35 to 45% more CPU than node2. Above that the two curves cross several times, because each server's CPU drops at a different load (node0 at 7000 and 12000 req/s per server, node2 at 10000 req/s per server). Near saturation node0 is the limit: 89.9% CPU at 38000 req/s, against 64.0% for node2. Past saturation node2 never rises above 67%, so it still had spare capacity when node0 ran out. Round-robin cannot use that capacity, because it sends node0 the same share no matter how loaded node0 is. Both the larger response and possibly other default-site settings make node0's requests more expensive, but the data cannot separate these causes.

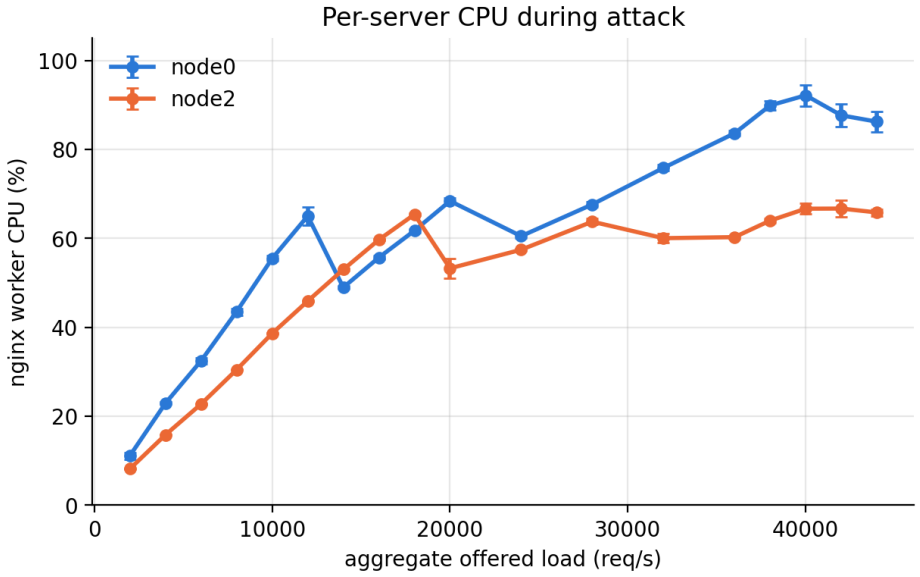


Figure 9: CPU of the nginx worker on each server.

Scaling

Metric	1 server (req/s)	2 servers (req/s)	Ratio
Saturation point	18000	38000	2.11
Capacity, p99 ≤ 1 ms	8000	16000	2.00
Capacity, p99 ≤ 10 ms	9000	18000	2.00
Capacity, p99 ≤ 100 ms	18000	38000	2.11

Adding a second server roughly doubled capacity on every measure. The ratio of 2.11 for the saturation point is a step size effect, not superlinear scaling. The single-node experiment tested 18000 and 19000 req/s, and 19000 req/s sat exactly on the edge (five of ten runs collapsed). In the multi-node experiment, 38000 req/s puts 19000 req/s on each server and all ten runs passed. The true capacity of one server lies between 18000 and 19000 req/s, so the ratio is close to 2.

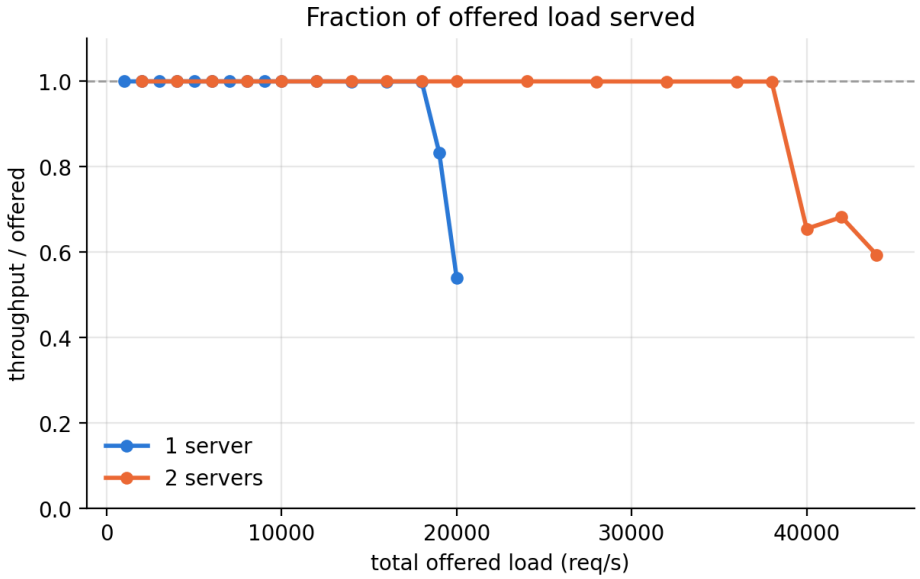


Figure 10: Fraction of the offered load that was served, for both setups.

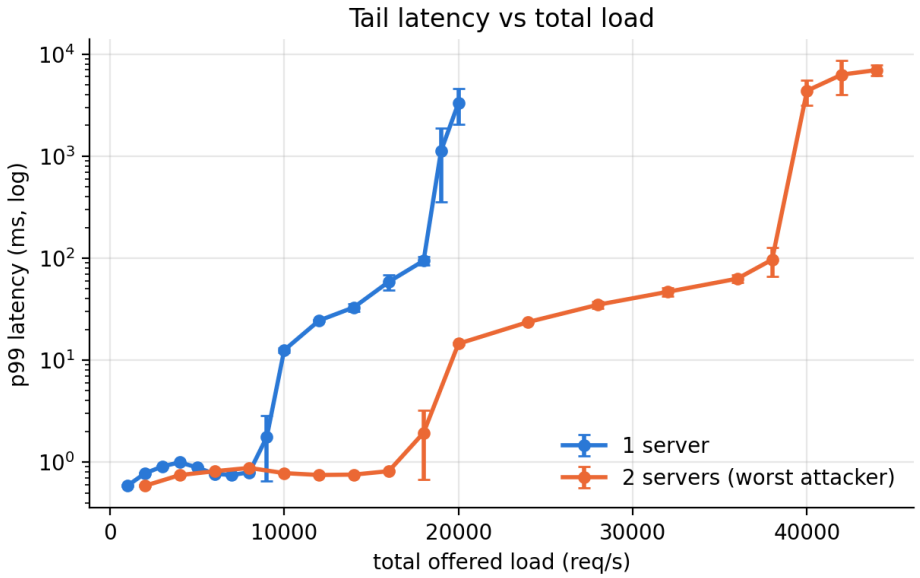


Figure 11: p99 latency against total offered load. The multi-node curve is shifted right by about a factor of two.

Servers do not interfere. If the two servers are independent, node0 at a given load per server should behave the same whether or not node2 is also serving. The table compares node0's CPU in both experiments at equal load per server.

Load per server (req/s)	node0 CPU, 1 server (%)	node0 CPU, 2 servers (%)	Difference (pp)	p (Welch)
1000	11.3	11.1	-0.1	0.780
2000	21.7	22.9	+1.2	0.004
3000	31.8	32.5	+0.7	0.426
4000	43.3	43.5	+0.2	0.659

Load per server (req/s)	node0 CPU, 1 server (%)	node0 CPU, 2 servers (%)	Difference (pp)	p (Welch)
5000	55.6	55.5	-0.1	0.817
6000	65.1	65.1	-0.1	0.964
7000	48.7	49.0	+0.3	0.230
8000	54.8	55.7	+0.9	0.003
9000	60.7	61.8	+1.1	<0.001
10000	66.5	68.4	+2.0	<0.001
12000	59.3	60.5	+1.2	<0.001
14000	67.0	67.6	+0.6	0.032
16000	76.2	75.9	-0.3	0.430
18000	84.6	83.6	-0.9	0.018
19000	91.8	89.9	-1.9	0.088
20000	75.8	92.2	+16.4	<0.001

Up to 19000 req/s per server the two curves differ by at most 2 percentage points. Some of these differences are statistically significant, because ten runs with very little run-to-run spread can detect even small gaps, but they are small in practice and change sign. The steps at 7000 and 12000 req/s appear in both. The one large gap, at 20000 req/s, is not a server effect. In the single-node run vegeta could not sustain the load and sent only 79% of the requested rate, so node0 received less traffic than the label says. Up to saturation, then, a server behaves the same whether it runs alone or next to another.

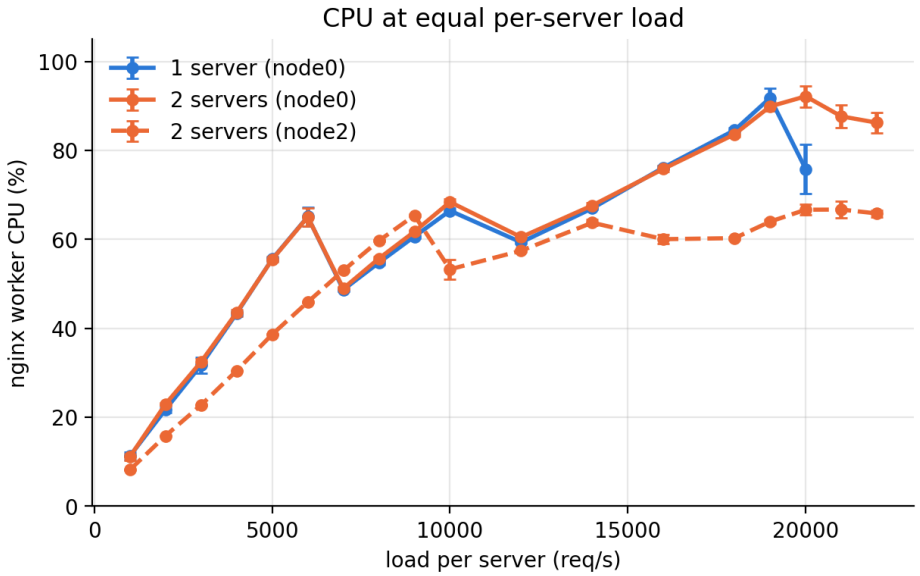


Figure 12: node0 and node2 CPU against load per server, with the single-node node0 curve for reference.

Conclusions

1. The node1 to node0 path carried UDP up to 900 Mbit/s with negligible loss. At 1000 Mbit/s three runs were lossless and seven lost 7 to 13%. The link is faster than 1 Gbit/s, so its capacity was not reached; the loss most likely occurs at the receiver.
2. One nginx worker saturates its CPU core at 18000 to 19000 req/s. The network carries about 93 Mbit/s of response bodies at that point, so the limit is the worker's core. Tail latency rises long before throughput suffers: p99 passes 10 ms at 10000 req/s.

3. Two servers with two attackers doubled every capacity measure: 38000 req/s at saturation, and 2 times the load at each p99 target. Scaling is close to linear because each server is limited by its own worker core, adding a server adds a core, and the servers do not affect each other's CPU cost.
4. With round-robin, the cluster is limited by its slowest server. node0, serving a larger default page, reached 90% CPU while node2 stayed at 64%, so node2's core was still about a third idle when the cluster saturated.

Limitations

- node0 served the 612-byte default nginx page instead of the 22-byte application page, so the single-node numbers and node0's share of the multi-node load do not use the same content as node2.
- node3's JSON reports were not saved. Its results come from the text reports in its logs, which have no timestamps, so node3's own CPU monitors are not used.
- Rates were tested in steps of 1000 to 4000 req/s, so each saturation point is only known to within one step.
- CPU was sampled once per second. CPU frequency, interrupt distribution and NIC statistics were not recorded, so the explanation for the stepwise drops in CPU cost is not verified.
- vegeta is open-loop. Past saturation it could not hold the requested rate, so results above saturation describe overload behaviour, not the response to a precise load.